

BraTS 2025 On-the-Fly 数据增强实现

Context

当前代码采用**离线预生成**策略：推理脚本遍历所有病例，为每个病例生成多个合成肿瘤并保存为 `.nii.gz` 文件，然后用增强后的数据集训练 nnU-Net。

BraTS 2025 要求改为 **on-the-fly 动态增强**：在 nnU-Net 训练的每个 batch 中，以 60% 概率选中样本，借入其他病人的真实标签，经过 SNFH→ET→NETC 级联转换和差分缩放后，通过扩散模型 inpainting 插入 1-2 个合成肿瘤，直接送入训练。

已有的基础设施（无需新建/修改）

- `model.py` (repo root)：提供 `make_beta_schedule()`（支持 linear/const/quad/jsd/sigmoid/cosine/cosine_reverse/cosine_anneal 共 8 种 schedule）、`diffusion_loss_fn()`、`sample_from_diff()` 等。`diffusion_inference_utils.py`、`tumour_main_diffusion.py`、`label_main_diffusion.py` 均已通过 `importlib` 正确导入，接口兼容。
- `network.py` (repo root)：1D/表格数据的扩散网络（`ConditionalLinear` 使用 FiLM 调制、`DiffusionModelWithEmbedding` 使用 `nn.Embedding` 时间条件）。不适用于 3D 图像，但提供了备选的时间调制参考模式。
- `diffusion_inference_utils.py`：已实现 3D DDPM 逆采样（`sample_label_diffusion`、`sample_tumour_diffusion_inpaint`）和系数构建（`make_diffusion_coefficients`），是 `model.py` 中 1D `sample_from_diff()` 的 3D 版本。
- `DiffusionNetwork.py`：已实现 `TimeConditionedSwinUNETR` 等时间条件 3D 网络，使用正弦嵌入 + 空间广播（比 `network.py` 的 `nn.Embedding` 更适合连续时间步）。

核心挑战

1. nnU-Net 以 patch 为单位训练，扩散模型需要 96^3 的子体积
2. nnU-Net 数据是 z-score 归一化的，扩散模型期望 $[-1, 1]$ min-max 范围（与 GAN 版本相同的归一化桥接策略，见下文）

3. nnU-Net 默认多进程 dataloader + GPU 模型需要特殊处理（通过禁用多进程增强解决，与 GAN spawn 策略等效）

改动概览

新建文件（4个）

1. `src/utils/label_modifier.py` — 借入标签修改器（SNFH→ET→NETC 级联 + 差分缩放）
2. `src/infer/on_the_fly_augmentation.py` — OnTheFlyTumourAugmenter 类（加载扩散模型 + 借入标签池 + 单样本增强）
3. `nnunetv2/training/data_augmentation/custom_transforms/on_the_fly_tumour.py` — batchgenerators AbstractTransform 包装器
4. `nnunetv2/training/nnUNetTrainer/variants/data_augmentation/nnUNetTrainerOnTheFly.py` — nnU-Net trainer 子类

修改文件（1个）

5. `src/infer/diffusion_inference_utils.py` — 添加 `rescale_to_mml()` / `rescale_from_mml()` 工具函数（用于 z-score ↔ [-1,1] 桥接）

保留不变的文件

- 所有 GAN 文件保持原样（`tumour_main.py`, `label_main.py`, `LabelGAN.py`, `Discriminator.py` 等）
- 扩散训练文件不变（`tumour_main_diffusion.py`, `label_main_diffusion.py`, `DiffusionNetwork.py`）
- `model.py` / `network.py` 不变（已与现有代码兼容）
- 所有数据工具函数不变（`data_utils.py`, `crop_label.py`, `gaussian_noise_tumour.py` 等）

详细设计方案

一、BraTS 2025 五步增强流程

对每个被选中的样本，执行以下步骤：

```
Step 1: 60% 概率选中该 batch 中的某个样本进行修改
Step 2: 从训练集标签池中随机借入一个其他病人的真实标签（含肿瘤区域）
Step 3: 对借入标签执行级联类替换（每步 70% 概率）：
        SNFH(2) → ET(3)，然后原始 ET(3) → NETC(1)
        （SNFH 改来的 ET 不再参与第二步转换）
Step 4: 基于 SNFH 是否被移除进行差分缩放：
        - 如果 SNFH 被移除（SNFH→ET 发生）：缩放系数 ∈ [0.1, 0.3]
        - 如果 SNFH 保留：缩放系数 ∈ [0.3, 0.8]
Step 5: 以 40% 概率插入第 2 个肿瘤；通过扩散 inpainting 生成肿瘤外观
```

二、label_modifier.py — 借入标签修改器

路径: Segmentation_Tasks/GliGAN/src/utils/label_modifier.py

```
def modify_borrowed_label(label, rng=None):
    """
    对借入的真实标签执行 BraTS 2025 的 Adapt Tumor Classes + Scale。

    输入: label (np.ndarray) — 整型标签，形状 (H, W, D)，值域 {0, 1, 2, 3, 4}
           1=NETC, 2=SNFH, 3=ET, 4=RC
    返回: modified_label (np.ndarray) — 修改后的整型标签（shape 不变）
           meta (dict) — {"snfh_removed": bool, "scale_factor": float, "changes": [...]}
    """
```

算法：

1. 提取 mask: `is_snfh = (label == 2)`, `is_et_original = (label == 3)`, `is_netc = (label == 1)`
2. 70% 概率: `label[is_snfh] = 3` (SNFH→ET), 记录 `snfh_removed = True`
3. 70% 概率: `label[is_et_original] = 1` (原始ET→NETC), 但不改变步骤2新生成的 ET
- 实现: 步骤2只修改 `is_snfh` 区域, 步骤3使用步骤2之前的 `is_et_original` mask
4. 缩放系数: `factor = rng.uniform(0.1, 0.3)` if `snfh_removed` else `rng.uniform(0.3, 0.8)`
5. 对标签非零区域做中心缩放: 计算肿瘤 bounding box, 以中心为锚点按 factor 收缩, 用 `scipy.ndimage.zoom` 或最近邻插值放回原尺寸

三、on_the_fly_augmentation.py — 核心增强器

路径: Segmentation_Tasks/GliGAN/src/infer/on_the_fly_augmentation.py

```
class OnTheFlyTumourAugmenter:
    """
    加载扩散模型 + 标签池，提供单样本 on-the-fly 增强。
    所有模型在 CPU 上运行（兼容 nnU-Net 多进程 dataloader）。
    """

    def __init__(self, diffusion_ckpt_dir, label_pool_paths, dataset_type,
                  n_steps=1000, beta_schedule="cosine", sampling_steps=50, device="cuda"):
        # 1. 加载 4 个模态的 TimeConditionedSwinUNETR 扩散模型到 GPU
        # - 从 diffusion_ckpt_dir/{tlce, tl, t2, flair}/weights/diffusion_{iter}.pt
        # 2. 加载 LabelDenoiser3D（暂不使用，保留以备后续无条件标签生成）
        # - 从 diffusion_ckpt_dir/label/weights/label_diffusion_{iter}.pt
        # 3. 加载标签池：直接加载所有 label 的 numpy 数组到内存 list
        # 4. 调用 make_diffusion_coefficients() 构建 beta/alphas_bar（均在 GPU 上）
        # 5. 设置采样步数（默认 50 步 DDPM 加速采样，而非完整 1000 步）

    def augment_sample(self, data_4ch, seg, rng=None):
        """
        输入：
        data_4ch: np.ndarray (4, D, H, W) — 4 模态，z-score 归一化
        seg: np.ndarray (1, D, H, W) — 原始分割标签（含 padding=-1）
        返回：
        data_4ch: np.ndarray (4, D, H, W) — 增强后
        seg: np.ndarray (1, D, H, W) — 合并了合成肿瘤的标签
        was_modified: bool
        """
```

核心流程：

```
if rng.uniform() >= 0.6: return (data_4ch, seg, False)

# 借入标签 + 修改
borrowed_label = label_pool[rng.choice(len(label_pool))] # numpy (H, W, D)
modified_label, meta = modify_borrowed_label(borrowed_label, rng)
# 上采样到与 data 相同的空间尺寸（borrowed_label 可能是其他分辨率）
modified_label = resize_label_to_match(modified_label, data_4ch.shape[1:])

# 插入第 1 个肿瘤
success = _insert_one_tumour(data_4ch, seg, modified_label, meta["scale_factor"], rng)
if not success: return (data_4ch, seg, False)

# 40% 概率插入第 2 个
if rng.uniform() < 0.4:
```

```

    borrowed_label_2 = label_pool[rng.choice(len(label_pool))]
    modified_label_2, meta_2 = modify_borrowed_label(borrowed_label_2, rng)
    modified_label_2 = resize_label_to_match(modified_label_2, data_4ch.shape[1:])
    _insert_one_tumour(data_4ch, seg, modified_label_2, meta_2["scale_factor"], rng)

return (data_4ch, seg, True)

```

`_insert_one_tumour()` 内部流程：

1. 在 `data_4ch[0]` (T1ce 模态) 中搜索空位：找 `modified_label` 非零但 `seg==0` 的位置，选随机中心
2. 以中心提取 96^3 子体积（不足 96 则跳过，pad 到 96）
3. 对每个模态的 96^3 patch: `rescale_array(patch, -1, 1)` → `add_gaussian_noise_tumour()` → `sample_tumour_diffusion_inpaint()` → `correct_background()` → `linear_interpolation()` → 逆 rescale 回原始值域 → 写回 `data_4ch`
4. 将 `modified_label` 的对应区域写入 `seg`

复用现有函数（从 inference 文件 import）：

- `add_gaussian_noise_tumour()` — 肿瘤区域加高斯噪声
- `correct_background()` — clamp 到 $[-1,1]$ ，背景设 -1
- `correct_label()` — 去掉脑外和与原有肿瘤重叠的标签
- `linear_interpolation()` + `get_inten_coord()` — 强度线性校正
- `rescale_array()` — min-max 归一化
- `sample_tumour_diffusion_inpaint()` — 扩散 inpainting (来自 `diffusion_inference_utils.py`)
- `make_diffusion_coefficients()` — 构建 $\beta/\alpha_{\text{bar}}$ (来自 `diffusion_inference_utils.py`)

函数向量化改造： 现有 `correct_background()`、`correct_label()` 等使用 Python 三重 for 循环遍历 96^3 (约 88 万个体素)，在 on-the-fly 场景下每次 batch 可能调用多次，需要向量化（用 numpy 布尔索引替代 for 循环），单次调用从 ~0.5s 降到 <0.01s。

四、on_the_fly_tumour.py — nnU-Net Transform 包装器

路径： `nnunetv2/training/data_augmentation/custom_transforms/on_the_fly_tumour.py`

```

from batchgenerators.transforms.abstract_transforms import AbstractTransform

class OnTheFlyTumourTransform(AbstractTransform):
    def __init__(self, augementer, data_key="data", seg_key="seg"):

```

```

self.augmenter = augmenter # OnTheFlyTumourAugmenter 实例
self.data_key = data_key
self.seg_key = seg_key

def __call__(self, **data_dict):
    data = data_dict[self.data_key] # (B, C, D, H, W) numpy float32
    seg = data_dict[self.seg_key] # (B, 1, D, H, W) numpy int16
    for b in range(data.shape[0]):
        data_b, seg_b, modified = self.augmenter.augment_sample(
            data[b:b+1], seg[b:b+1]
        )
        if modified:
            data[b] = data_b[0]
            seg[b] = seg_b[0]
    data_dict[self.data_key] = data
    data_dict[self.seg_key] = seg
    return data_dict

```

插入位置： 在 `get_training_transforms()` 中作为**第一个 transform**（SpatialTransform 之前）。这样数据仍在原始方向，扩散模型看到的是未经旋转的 patch，生成的肿瘤随后会被所有后续增强（旋转、缩放、噪声、模糊等）一起变换。

五、nnUNetTrainerOnTheFly.py — nnU-Net Trainer 子类

路径： `nnunetv2/training/nnUNetTrainer/variants/data_augmentation/nnUNetTrainerOnTheFly.py`

```

from nnunetv2.training.nnUNetTrainer.nnUNetTrainer import nnUNetTrainer
from nnunetv2.training.data_augmentation.custom_transforms.on_the_fly_tumour import OnTheFlyTumourTransform
from Segmentation_Tasks.GliGAN.src.infer.on_the_fly_augmentation import OnTheFlyTumourAugmenter

class nnUNetTrainerOnTheFly(nnUNetTrainer):
    def __init__(self, plans, configuration, fold, dataset_json, ...):
        super().__init__(plans, configuration, fold, dataset_json, ...)
        # 从环境变量读取扩散模型路径和标签池路径
        diffusion_ckpt_dir = os.environ.get("DIFFUSION_CKPT_DIR",
            "Segmentation_Tasks/GliGAN/Checkpoint/brats_2024")
        label_pool_glob = os.environ.get("LABEL_POOL_GLOB",
            "nnUnet_raw/DatasetXXX_BraTS/imagesTr/*_seg.nii.gz")
        self.augmenter = OnTheFlyTumourAugmenter(
            diffusion_ckpt_dir=diffusion_ckpt_dir,
            label_pool_paths=sorted(glob(label_pool_glob)),
            dataset_type="BRATS_2023",
            sampling_steps=50,

```

```

        device="cuda" if torch.cuda.is_available() else "cpu"
    )

def get_training_transforms(self, *args, **kwargs) -> AbstractTransform:
    # 在父类 transform chain 最前面插入 OnTheFlyTumourTransform
    # 覆盖为实例方法以访问 self.augmenter
    ...
    tr_transforms = [
        OnTheFlyTumourTransform(self.augmenter),
        ... # nnUNetTrainer.get_training_transforms 的其余 transforms
        NumpyToTensor(['data', 'target'], 'float'),
    ]
    return Compose(tr_transforms)

def get_dataloaders(self):
    # 强制单线程增强器以安全使用 GPU（避免多进程 CUDA fork 问题）
    # 原 GAN 版本用 spawn 解决此问题；这里用 SingleThreadedAugmenter 等效
    self._original_get_allowed_n_proc = get_allowed_n_proc_DA
    # 临时覆盖：强制返回 0
    import nnunetv2.training.nnUNetTrainer.nnUNetTrainer as _t
    _t.get_allowed_n_proc_DA = lambda: 0
    dl = super().get_dataloaders()
    _t.get_allowed_n_proc_DA = self._original_get_allowed_n_proc
    return dl

```

GPU 推理原理：模型在 Augmenter 初始化时加载到 GPU，transform 在 `SingleThreadedAugmenter` 主线程中运行，直接调用 GPU 模型。不需要 pickle/CUDA spawn，与 nnU-Net 多进程 dataloader 的数据预取兼容（dataloader 子进程只做 I/O，不做 transform）。

使用方法：

```

export DIFFUSION_CKPT_DIR="Segmentation_Tasks/GliGAN/Checkpoint/brats_2024"
export LABEL_POOL_GLOB="nnUNet_raw/Dataset232_BraTS/imagesTr/*_seg.nii.gz"
nnUNetv2_train 232 3d_fullres 0 -tr nnUNetTrainerOnTheFly

```

六、环境配置变化

零变化。 所有运行时依赖已存在于现有环境中：

依赖	来源
`torch`,`numpy`,`scipy`,`pandas`	conda 环境
`monai` (SwinUNETR, Resize, EnsureChannelFirst)	`pip install -r requirements_seg.txt`
`batchgenerators` (AbstractTransform, Compose)	nnU-Net 依赖
`nibabel`	nnU-Net 依赖

不需要新增任何 pip/conda 包。`model.py` 和 `network.py` 在仓库根目录，已被 diffusion 代码正确导入。

七、GAN 代码处理

所有 GAN 文件保留在原位（`tumour\_main.py`,`label\_main.py`,`LabelGAN.py`,`Discriminator.py` 等），不被 on-the-fly 流程引用。已存在的 `\_NOT\_USED` 备份文件也保持不变。运行时仅调用扩散模型。

八、关键设计决策

- 1. **GPU 推理（与原 GAN 一致）**：扩散模型加载到 GPU，通过强制 `allowed\_num\_processes=0` 使用 `SingleThreadedAugmenter`，避免多进程 CUDA fork 问题。原 GAN 用 `spawn` 策略解决此问题，这里用单线程增强器达到同等效果。dataloader 子进程（I/O 预取）与 transform（GPU 推理）分离不受影响。96³ 体积、50 步加速采样，单次 inpainting 约 0.5-1 秒（GPU）vs 2-5 秒（CPU）。
- 2. **归一化桥接（完全复用原 GAN 逻辑）**：`rescale\_array(arr, -1, 1)` → 扩散模型 → `correct\_background()` → `get\_inten\_coord()` → `linear\_interpolation()`。此流程与原 GAN `insert\_tumour()` 第 471-486 行完全一致，仅 `linear\_interpolation` 映射的目标值域从原始扫描值变为 z-score 值。全部函数直接 import 复用，无需重写。原理相同：都是将模型输出的 [-1,1] 值映射回宿主扫描的实际强度分布。
- 3. **标签池**：在 Augmenter 初始化时加载所有训练样本的 label `.nii.gz` 文件到内存（单通道 int16，约 240×240×155 ≈ 9MB/样本，100 个样本 ≈ 900MB）。对于大数据集，可改为按需加载或只加载 bounding box 内的肿瘤区域（大幅减小内存）。

4. **Patch 尺寸约束**: 如果 nnU-Net patch 在任一维度 < 96 , 跳过该样本的增强。大多数 nnU-Net 3D patch (如 $128 \times 160 \times 112$) 满足此条件。
5. **向量化后处理**: 现有 `correct_background()`、`correct_label()` 使用三重 for 循环, 在 on-the-fly 中需向量化为 numpy 布尔索引操作, 避免成为瓶颈。
6. **采样步数权衡**: 使用 DDPM 加速采样 (从 1000 步减到 50 步), 在质量和速度间取得平衡。可通过调整 `sampling_steps` 控制。

验证方法

1. **单元测试**: `python -c "from src.utils.label_modifier import modify_borrowed_label; ..."` — 验证级联逻辑和缩放
2. **增强器测试**: 用 1 个真实病例运行 `augment_sample()`, 保存输出 `.nii.gz`, 人工检查肿瘤外观和标签
3. **Transform 集成**: 构造假 `data_dict` (随机 numpy array), 测试 `OnTheFlyTumourTransform`
4. **训练测试**: 用小数据集 (10 个病例) 跑 1 epoch nnU-Net 训练, 确认 loss 下降、无 OOM
5. **环境测试**: 确认不需要新 pip install